
GramVest

SIH 2026 • PS 26091

Integration Playbook - How the 5 Modules Combine

Shared team contract for module integration

TEAM IMPLEMENTATION HANDBOOK

Prepared as a practical engineering guide for independent module development, stable integration, testing, and handoff.

Source basis: SIH 26091 Implementation Blueprint and the corresponding GramVest module Markdown handbook.

Contents

1 Purpose	1
2 1. Team map	1
3 2. The shared object	1
4 3. Integration contract rules	1
4.1 Rule 1	1
4.2 Rule 2	1
4.3 Rule 3	1
4.4 Rule 4	2
4.5 Rule 5	2
4.6 Rule 6	2
5 4. Step-by-step development workflow	2
5.1 Stage 0 — scope freeze	2
5.2 Stage 1 — foundation	2
5.3 Stage 2 — finance	2
5.4 Stage 3 — geo/market	3
5.5 Stage 4 — intelligence	3
5.6 Stage 5 — RAG/advisor	3
5.7 Stage 6 — full UI integration	3
5.8 Stage 7 — What-If + DPR	3
5.9 Stage 8 — validation/demo	4
6 5. API ownership	4
7 6. Shared response metadata	4
8 7. Demo story	4
9 8. Example end-to-end payload	5
10 9. Integration testing	5
10.1 Test A — normal case	5
10.2 Test B — missing competitor data	5
10.3 Test C — low capital	6
10.4 Test D — high costs	6
10.5 Test E — scheme cap	6
10.6 Test F — AI missing data	6
10.7 Test G — What-If	6
10.8 Test H — report consistency	6
11 10. Git strategy	6
12 11. Definition of done — whole product	6
13 12. Security and trust	7
14 13. What not to build for MVP	7
15 14. Final presentation rule	7

Purpose

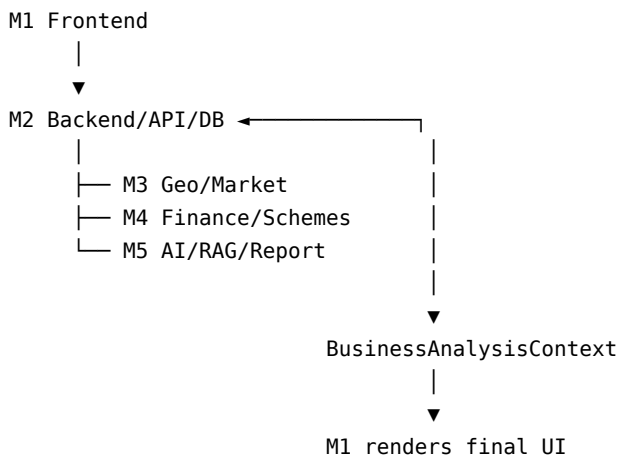
This file is the common team contract. Every member should read it before building integrations.

The core goal is simple:

Each module works independently, but all modules exchange predictable data through stable APIs and shared schemas.

The end product must behave as one guided journey, not five separate projects.

1. Team map



2. The shared object

Create one canonical domain object.

```
{
  "profile": {},
  "location": {},
  "market": {},
  "competitors": {},
  "opportunity": {},
  "risks": {},
  "viability": {},
  "finance": {},
  "schemes": {},
  "advisor": {},
  "scenarios": [],
  "report": {}
}
```

This is a logical object; actual persistence can be split across tables.

3. Integration contract rules

Rule 1

Do not change a shared field name without a team decision.

Rule 2

Do not remove a field used by another module without versioning the contract.

Rule 3

Do not put authoritative calculations in frontend.

Rule 4

Do not put scheme rules in frontend.

Rule 5

Do not let AI become the source of financial truth.

Rule 6

Every major estimate carries confidence/source/assumptions when applicable.

4. Step-by-step development workflow

Stage 0 — scope freeze

Team jointly decides:

- demo geography
- 2–3 business categories
- core persona
- MVP features
- database entities
- API names

Stage 1 — foundation

M2:

- repo
- DB
- migrations
- API schemas

M1:

- app shell
- navigation
- mock-data pages

M3:

- data folder structure
- geography seed

M4:

- finance package + tests

M5:

- AI folder + prompt/RAG skeleton

Stage 2 — finance

M4 finishes:

- project cost
- EMI
- cash flow
- P&L
- DSCR
- break-even
- scheme configuration

M2 exposes endpoints.

M1 connects Money.

Stage 3 — geo/market

M3:

- geocoding
- PostGIS
- 5/10 km
- market reach
- competitors
- opportunity evidence

M2 exposes API.

M1 connects Market/Opportunity.

Stage 4 — intelligence

M4:

- viability score

M3:

- opportunity evidence

M5:

- SWOT/risk explanations

Stage 5 — RAG/advisor

M5:

- document ingestion
- chunking
- retrieval
- grounded advisor
- guards

Stage 6 — full UI integration

M1:

- remove mocks
- connect all live endpoints
- add loading/error/empty states

Stage 7 — What-If + DPR

M4:

- scenario calculation

M5:

- report generation

M1:

- scenario UI + report UI

Stage 8 — validation/demo

Everyone:

- test full flow
- fix integration issues
- freeze demo
- record fallback

5. API ownership

Endpoint	Owner	Consumer
POST /api/profile	M2	M1
GET /api/location/resolve	M2 + M3	M1
POST /api/market/analyze	M2 + M3	M1, M5
GET /api/market/competitors	M2 + M3	M1
POST /api/opportunity/score	M2 + M3/M4	M1, M5
POST /api/finance/project	M2 + M4	M1, M5
POST /api/finance/repayment	M2 + M4	M1, M5
POST /api/finance/dscr	M2 + M4	M1, M5
POST /api/schemes/route	M2 + M4	M1, M5
POST /api/advisor/chat	M2 + M5	M1
POST /api/report/generate	M2 + M5	M1
POST /api/simulator/what-if	M2 + M4	M1, M5

6. Shared response metadata

All module outputs should preserve:

```
source
source_date
version
confidence
assumptions
```

Use only fields that apply.

7. Demo story

Use one realistic entrepreneur story throughout the demo.

Suggested flow from the blueprint:

1. Enter location

2. Enter capital
3. Select business
4. Analyze 5/10 km market
5. Show competitors
6. Identify local gap
7. Show SWOT/risk
8. Explain viability score
9. Structure project/loan
10. Show indicative scheme route
11. Show EMI/cash flow/DSCR
12. Ask AI advisor
13. Change a What-If assumption
14. Regenerate recommendation
15. Download DPR

8. Example end-to-end payload

```
{
  "profile": {
    "business": "Dairy Processing Unit",
    "location": "Example Village, Jalandhar",
    "own_capital": 300000
  },
  "market": {
    "potential_customers": 1210,
    "households": 3920,
    "competitor_count": 7
  },
  "opportunity": {
    "score": 76
  },
  "viability": {
    "score": 74
  },
  "finance": {
    "project_cost": 500000,
    "financing_required": 200000,
    "monthly_revenue": 85000,
    "monthly_expenses": 63000,
    "monthly_surplus": 22000,
    "emi": 3119,
    "dscr": 1.51
  },
  "schemes": {
    "recommended": "indicative route"
  }
}
```

These numbers are demonstration values only.

9. Integration testing

Test A — normal case

New user → successful market → finance → report.

Test B — missing competitor data

Market page must show low confidence rather than zero competition.

Test C — low capital

Finance must detect increased financing need.

Test D — high costs

Cash flow/DSCR/viability update.

Test E — scheme cap

Scheme route should change when project exceeds configured limits.

Test F — AI missing data

AI refuses to invent a number.

Test G — What-If

Change demand → financial result → viability → advisor explanation.

Test H — report consistency

Numbers on dashboard, finance page, AI explanation and PDF match.

10. Git strategy

Branches:

```
main
develop
feature/frontend
feature/backend
feature/geo-market
feature/finance
feature/ai-rag
```

Rule:

```
code
→ local test
→ pull request
→ review
→ develop
→ integration test
```

Do not merge directly to main during normal development.

11. Definition of done — whole product

```
[ ] scope frozen
[ ] database schema finalized
[ ] API contracts finalized
[ ] financial formulas tested
[ ] scheme rules versioned
[ ] demo geography selected
[ ] demo businesses prepared
[ ] PostGIS radius query works
[ ] market/competitor API works
[ ] opportunity score documented
[ ] risk model works
[ ] viability score works
[ ] RAG knowledge base created
[ ] AI guardrails implemented
[ ] UI connected to APIs
```

[] interactive map works
[] What-If works
[] PDF generated
[] end-to-end journey tested
[] fallback demo prepared

12. Security and trust

Follow these team rules:

- collect only required prototype information
- do not request unnecessary Aadhaar/sensitive identity information
- separate user profile data from public market data
- validate APIs server-side
- never expose API keys in frontend
- log calculation assumptions/version where useful
- show confidence when data is incomplete
- label scheme routes as indicative
- clearly distinguish advisory output from official eligibility/sanction

13. What not to build for MVP

Do not spend core sprint time on:

- voice interface
- WhatsApp/SMS
- offline-first
- direct bank API integration
- direct government portal integration
- advanced demand forecasting
- long-term monitoring
- large-scale live ingestion

Present these as future roadmap items unless the core MVP is already stable.

14. Final presentation rule

Do not present five modules separately for the entire demo.

Present one entrepreneur story.

Each module should appear as the system moves through the journey.

That makes the architecture understandable:

M3 provides evidence. M4 calculates. M5 explains. M2 connects. M1 presents.